

Parallel Breadth-First Search: GPU Linear Algebra, Graph-First Traversal, and Distributed MPI Partitioning

Aakaash Nattanmai
Cornell University
Ithaca, NY, USA

Harshaan Chugh
Cornell University
Ithaca, NY, USA

ABSTRACT

Breadth-first search (BFS) is a core graph primitive, but its irregular frontiers and skewed memory access patterns make it difficult to optimize on modern parallel systems. This project compares three BFS design spaces: linear-algebraic GPU BFS, graph-first direction-optimizing GPU BFS, and distributed-memory MPI BFS. On a single NVIDIA A100, we implement five linear-algebraic sparse-matrix-vector-style BFS variants and a graph-first push/pull traversal. We also add an optional replicated-CSR multi-GPU prototype to test whether extra GPUs help without communication-aware frontier exchange. On Perlmutter CPU nodes, we extend the project with 1D and 2D MPI graph partitioning, bitmap frontier reductions, and SNAP road network benchmarks. The fastest SpMV-style GPU variant wins on all six synthetic GPU inputs, with graph-first within 1.14–1.33x on random sparse/dense graphs but 2.69x slower on the power-law graph. In MPI, 2D partitioning improves the dense synthetic graph at 16 ranks, while 1D bitmap frontier reductions are consistently better on sparse road networks up to 64 ranks. The multi-GPU prototype shows negative scaling on 2 and 4 GPUs, highlighting the cost of per-level frontier synchronization. These results suggest that no single BFS formulation dominates: the best design depends on whether memory bandwidth, frontier construction, communication volume, or graph diameter is the limiting cost.

1 INTRODUCTION

BFS is a building block for shortest paths, connected components, centrality, web crawling, and many graph analytics pipelines. It is also a representative stress test for high-performance computing: each level depends on the previous frontier, work per vertex is highly irregular, and memory accesses follow graph structure rather than dense array order. On GPUs, this creates warp divergence, load imbalance, and atomic update pressure. On distributed-memory systems, it turns into a communication problem because each frontier may contain vertices owned by many ranks.

Our project asks how the choice of abstraction affects BFS performance. We started from the course suggestion to compare a graph-first BFS with a linear-algebraic formulation inspired by GraphBLAS. We then extended the study after the poster checkpoint with MPI implementations that compare 1D and 2D partitioning. Our hypothesis was that linear-algebraic GPU kernels would be competitive when frontier operations could be expressed as regular sparse row scans, while graph-first traversal would benefit when frontier sizes were small enough to avoid scanning most vertices. For MPI, we expected 2D partitioning to reduce communication at high rank counts, especially on graphs with enough edges to amortize its extra setup.

The project contributions are:

- five CUDA BFS kernels: baseline, warp-cooperative, bitmap, push-pull, and fused warp-bitmap;
- a CUDA graph-first BFS with top-down push, bottom-up pull, bitmap frontiers, and Beamer-style alpha/beta switching;
- a replicated-CSR multi-GPU BFS prototype with 1, 2, and 4 GPU measurements;
- 1D and 2D MPI BFS implementations with bitmap frontiers, bitwise-OR reductions, and SNAP road-network support;
- correctness validation against a Python BFS and benchmark results on synthetic and real road graphs.

2 RELATED WORK

Beamer et al. introduced direction-optimizing BFS [1], which motivates the push/pull split in our graph-first CUDA implementation. In top-down mode, frontier vertices push discoveries to neighbors; in bottom-up mode, unvisited vertices pull by checking whether any neighbor is in the frontier. Merrill et al. [2] showed that GPU BFS needs careful frontier management, load balancing, and prefix/bitmap techniques to avoid underutilization. Our warp-cooperative and bitmap variants are simplified experiments in that design space.

The linear-algebraic framing is motivated by GraphBLAS and related work that casts graph traversal as sparse linear algebra over semirings [3]. The MPI extension is motivated by distributed sparse matrix and graph partitioning work, especially 2D sparse matrix distributions used in CombBLAS [4]. Our SNAP road-network inputs follow the SNAP graph collection [5].

3 IMPLEMENTATIONS

3.1 Common Graph Representation

All GPU variants use the same host-side graph loader and CSR representation: `row_ptr[V+1]` and `col_idx[E]`. Synthetic graph files are stored as undirected edge lists and loaded with both directions inserted. The MPI loader supports both the project `.edgelist` format and SNAP `.txt` files. Each implementation computes integer BFS depths from a fixed source vertex and can dump full depth vectors for validation.

3.2 Linear-Algebraic GPU BFS

The linear-algebraic implementation expresses each BFS level as a masked sparse matrix-vector operation

$$f_{k+1} = (Af_k) \odot \neg v_k,$$

where f_k is the frontier and v_k is the visited mask. We implemented five kernel variants.

The **baseline** assigns one CUDA thread per vertex and scans that row for any neighbor in the current frontier. **Warp-cooperative** processing assigns one warp per row, so high-degree rows are split across 32 lanes and combined with a warp vote. The **bitmap** variant compresses the frontier from one integer per vertex to one bit per vertex, reducing bandwidth and cache pressure. **Push-pull** adds a frontier-driven push kernel for small frontiers and switches to pull when an estimated edge budget crosses a Beamer threshold. Finally, **warp-bitmap** combines warp-cooperative row scans with bitmap frontier checks.

3.3 Graph-First GPU BFS

The graph-first CUDA implementation uses frontier queues and bitmap frontiers. The top-down kernel assigns one thread to each frontier vertex and uses `atomicCAS` on `visited[]` to prevent duplicate discoveries. The bottom-up kernel assigns one thread to each unvisited vertex and stops as soon as it finds a neighbor in the frontier bitmap. The host maintains the frontier size and frontier edge count each level. It switches from top-down to bottom-up when $m_f > |E|/\alpha$ and switches back when $|f_k| < |V|/\beta$, using $\alpha = 14$ and $\beta = 24$ by default.

3.4 Multi-GPU Linear-Algebraic Prototype

The optional multi-GPU extension keeps the linear-algebraic bitmap-pull formulation but partitions the output rows across multiple GPUs on one Perlmutter node. Each GPU stores a full CSR copy and owns a contiguous vertex range for writing the next frontier. At every BFS level, the host broadcasts the current frontier bitmap to all GPUs, launches the row-partitioned pull kernel on each device, copies each partial next-frontier bitmap back to the host, ORs those partial bitmaps together, and repeats. This replicated-CSR prototype is simple and useful as a scaling experiment, but it is intentionally not a production multi-GPU BFS: it pays a full host-mediated frontier exchange every level and does not use peer-to-peer GPU communication or a distributed CSR layout.

3.5 MPI BFS

The post-poster extension adds two CPU-only MPI BFS implementations. The 1D version partitions vertices into contiguous row blocks. Each rank scans its locally owned unvisited vertices using a global frontier bitmap, then contributes new discoveries through `MPI_Allreduce` with `MPI_BOR`. The bitmap representation makes each frontier a fixed-size $|V|/8$ byte buffer and maps naturally to set union. Early termination is implemented by popcounting the merged frontier.

The 2D version arranges ranks into a $\sqrt{P} \times \sqrt{P}$ grid and assigns each rank a block of the adjacency matrix. Each rank extracts the frontier slice for its column block, scans its local submatrix, and OR-reduces newly discovered row-block vertices within its process row. For simplicity, the implementation still keeps a full global frontier and depth vector on each rank and performs a global OR to assemble the next frontier. This makes the 2D implementation a useful partitioning experiment, but not a fully memory-minimal distributed BFS.

4 COST MODEL

BFS performance can be separated into per-level work and per-level synchronization. On the GPU, a simplified traversal time is

$$T_{\text{GPU}} \approx Lt_{\text{launch}} + \sum_{k=1}^L (M_k/B_{\text{mem}} + A_k t_{\text{atomic}}),$$

where L is the number of BFS levels, M_k is the memory traffic for level k , and A_k is the number of atomic updates. This explains why a chain graph is slow despite having few edges: it has many levels and very little parallel work per launch. It also explains why power-law graphs stress graph-first push: high-degree hubs can create many atomic queue updates in the same level.

For the MPI bitmap BFS, let $W = \lceil |V|/64 \rceil$ be the number of 64-bit bitmap words and P the number of ranks. The 1D implementation performs one global OR-reduction over W words per level, so an alpha-beta model gives

$$T_{1D,k} \approx C_k/P + \alpha \log P + 8W\beta \log P,$$

where C_k is local edge-scan work. The 2D prototype adds a row reduction over $W/\sqrt{P}$ words, but still assembles a full global frontier:

$$T_{2D,k} \approx C_k/P + \alpha \log \sqrt{P} + 8(W/\sqrt{P})\beta \log \sqrt{P} + \alpha \log P + 8W\beta \log P.$$

Thus 2D can win when the reduced local edge work and row-level collectives matter, but the current prototype still pays a global-frontier cost. This model matches the experiments: 2D helps most on the dense synthetic graph and least on sparse road networks with many low-work levels.

5 EXPERIMENTAL METHODOLOGY

GPU experiments ran on one NVIDIA A100 on NERSC Perlmutter with `CUDA_VISIBLE_DEVICES=0`. Timings use CUDA events around the traversal only, excluding graph load and output. Each reported GPU number is the median of 10 runs. Optional multi-GPU experiments ran on one 4-GPU Perlmutter node and report the median of 3 runs for 1, 2, and 4 requested GPUs. Those timings include the prototype’s host-mediated bitmap synchronization between levels. MPI experiments ran on Perlmutter CPU nodes with 10 trials for synthetic graphs and 3 trials for SNAP road networks. MPI timings use `MPI_Wtime` around the BFS traversal.

We used six synthetic inputs: medium graphs with 10^3 vertices and large graphs with 10^5 vertices, covering chain, dense random, sparse random, and power-law structure. We also evaluated SNAP road networks: roadNet-PA, roadNet-TX, and roadNet-CA. Correctness was checked on tiny graphs against a Python BFS. For SNAP, we checked that 1D runs with different rank counts and the 2D implementation produced matching depth vectors on the same source.

6 GPU RESULTS

Figure 1 and Table 1 show that the best linear-algebraic variant wins on all six current GPU inputs. This does not mean that a generic SpMV formulation is always faster; rather, it reflects the particular costs of our graph-first implementation. Graph-first must construct new queues, maintain queue and bitmap representations, and use atomics during push. On random sparse and dense graphs, the gap is modest: 1.14–1.33x on medium/large sparse and large dense. The

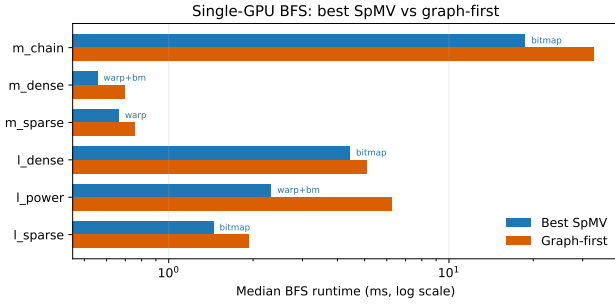


Figure 1: Single-GPU A100 runtime comparison. Each blue bar is the fastest linear-algebraic SpMV variant for that graph; orange is graph-first direction-optimizing BFS. The x-axis is logarithmic.

Table 1: Median GPU BFS time in milliseconds. The best SpMV variant is selected per graph and compared with graph-first direction optimization.

Graph	Best SpMV	SpMV	Graph-first	Ratio
medium_chain	bitmap	18.733	32.918	1.76x
medium_dense	warpbitmap	0.557	0.697	1.25x
medium_sparse	warp	0.665	0.758	1.14x
large_dense	bitmap	4.438	5.085	1.15x
large_powerlaw	warpbitmap	2.325	6.264	2.69x
large_sparse	bitmap	1.446	1.927	1.33x

Table 2: Optional replicated-CSR multi-GPU BFS results. Speedup is relative to the same prototype using one GPU, so values below 1.0 indicate slowdown.

Graph	1 GPU ms	2 GPU speedup	4 GPU speedup
large_dense	0.550	0.93x	0.65x
large_powerlaw	0.599	0.90x	0.72x
large_sparse	0.403	0.67x	0.41x
roadNet-PA	50.029	0.69x	0.40x
roadNet-TX	78.738	0.72x	0.40x
roadNet-CA	76.352	0.71x	0.40x

large power-law graph is the hardest case for graph-first because hub-heavy frontiers amplify queue and atomic overhead. The chain graph is pathological for both paradigms because the BFS depth is 999, forcing many sequential kernel launches with little parallel work per level.

The SpMV variants also show that optimizations are topology-dependent. Bitmap frontiers improve bandwidth pressure on large dense and sparse graphs. The warp-bitmap fusion is best on the large power-law graph, where high-degree hubs benefit from cooperative row scanning. Push-pull is not universally helpful in the SpMV code because its switching heuristic uses an average-degree estimate; on some inputs, the extra bookkeeping outweighs the saved work.

Table 2 shows that the naive multi-GPU prototype does not scale: 2 GPUs are slower than 1 GPU on every input, and 4 GPUs are slower still. This is an informative negative result. The per-level bitmap exchange is small in absolute bytes, but BFS performs one exchange per level, and the prototype routes every exchange through host memory. On road networks, this overhead is especially visible because roadNet-PA, roadNet-TX, and roadNet-CA require 541, 722, and 554 BFS levels respectively. Extra GPUs reduce the number of output rows scanned per device, but the saved row-scan work is smaller than the added synchronization and device-management cost for these graphs.

7 MPI RESULTS

The MPI results answer a different question than the GPU results: how BFS partitioning interacts with communication. On the large dense synthetic graph, 2D partitioning has the best high-rank result, reaching 0.003220 s at 16 ranks compared with 0.006686 s for 1D. This supports the expected advantage of 2D partitioning when enough edges exist to amortize submatrix setup and when row reductions reduce per-rank work.

The power-law and sparse synthetic graphs favor 1D. On large power-law, 1D improves from 0.005864 s at one rank to 0.001485 s at 16 ranks, while 2D improves from 0.006811 s to 0.002615 s. On large sparse, the two strategies are close at 16 ranks. These results suggest that our 2D implementation is most useful when edge work is high; otherwise, the global-frontier simplification and extra reductions dominate.

SNAP road networks strengthen this conclusion. Table 4 shows that 1D reaches roughly 13–14x speedup at 64 ranks on all three road networks. For roadNet-CA, 1D improves from 5.53 s at one rank to 0.41 s at 64 ranks, while 2D improves from 7.82 s to 1.61 s. 1D also wins on roadNet-PA and roadNet-TX at all tested rank counts. Road graphs have low average degree and high diameter, so each level has relatively little edge work and many synchronization rounds. The fixed global bitmap reduction in 1D is simple and efficient enough that the current 2D prototype’s extra communication is not recovered.

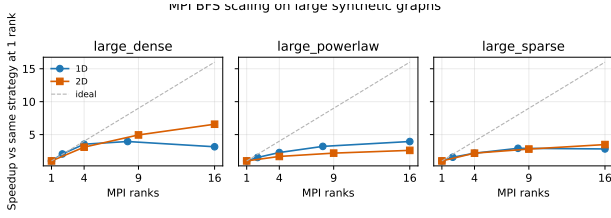


Figure 2: MPI speedup on the large synthetic graphs. Speedup is normalized against the same strategy at one rank.

Table 3: Large synthetic MPI results at 16 ranks. Speedup is relative to the same strategy at one rank.

Graph	1D time	1D speedup	2D time	2D speedup
large_dense	0.006686	3.15x	0.003220	6.57x
large_powerlaw	0.001485	3.95x	0.002615	2.60x
large_sparse	0.003157	2.82x	0.003262	3.48x

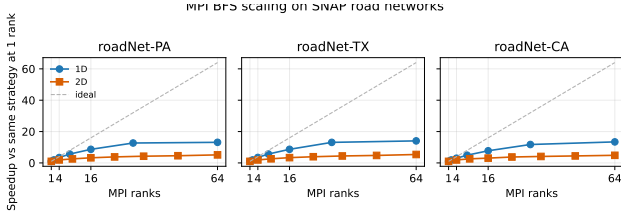


Figure 3: MPI speedup on SNAP road networks. 1D partitioning scales better than the current 2D prototype on these sparse high-diameter road graphs.

Table 4: SNAP road-network MPI results at 64 ranks. 1D speedup is relative to the 1D one-rank runtime.

Graph	1D 64-rank time	1D speedup	2D 64-rank time
roadNet-PA	0.243842	13.14x	0.863500
roadNet-TX	0.383658	14.06x	1.390648
roadNet-CA	0.412492	13.41x	1.611111

8 VALIDATION AND REPRODUCIBILITY

Correctness was checked in three ways. First, all tiny edgelist graphs were validated against `test-graphs/validate_bfs.py`. Second, the GPU benchmark harness can pass `-dump` to emit one depth per vertex for comparison. Third, the MPI SNAP tests compared depth vectors across 1D and 2D executions rather than checking only aggregate counts. This is important because two BFS implementations can report the same number of reachable vertices while still disagreeing on levels.

All benchmark commands are scripted. GPU experiments use the Python benchmark driver in `scripts/`; MPI experiments use the Slurm batch scripts in the same directory. The report figures are regenerated by `scripts/plot_report_results.py` from committed CSV files, which keeps the PDF plots tied to the raw data. Build

instructions and result paths are documented in the repository README.

9 DISCUSSION

The main lesson is that BFS optimizations are strongly graph- and architecture-dependent. On a single GPU, dense bitmap and warp-cooperative frontier representations are valuable because they reduce bandwidth and load imbalance. However, a graph-first implementation needs careful queue construction to compete with row-scan SpMV kernels. In distributed-memory MPI, the same BFS problem becomes a frontier communication problem: bitmap OR reductions are simple and robust, while 2D partitioning only helps when communication reduction exceeds the cost of additional coordination. The multi-GPU experiment reinforces the same point on a single node: simply adding devices is not enough if every level performs a centralized frontier merge.

Our study has limitations. The GPU implementations are handwritten CUDA kernels rather than cuGraph or a production GraphBLAS backend. The MPI implementations are CPU-only and load graphs by broadcasting edge arrays from rank 0, which is adequate for our project graphs but not scalable to Graph500-sized inputs. The 2D MPI implementation keeps global frontier and depth arrays for simplicity, so it tests a 2D work partition but not the full memory savings of a distributed 2D BFS. The multi-GPU prototype similarly keeps a replicated CSR on every GPU and synchronizes through the host, so it should be read as a communication-cost probe rather than a tuned multi-GPU BFS. Finally, GPU and MPI timings should not be interpreted as direct hardware comparisons; they answer different design questions.

10 FUTURE WORK

The most immediate improvement is to make the graph-first GPU BFS more competitive by reducing queue construction overhead. A warp-aggregated push kernel could reserve queue slots per warp rather than per discovered vertex, reducing atomic contention on power-law graphs. For the SpMV code, the push-pull heuristic should use the actual frontier edge count rather than an average-degree estimate, matching the graph-first driver.

For MPI, a stronger 2D implementation would avoid storing the full frontier on every rank. The diagonal owner of each column block should broadcast frontier slices down process columns, and row owners should assemble only their partition of the next frontier. This would better realize the $O(\sqrt{P})$ communication model. For multi-GPU BFS, the corresponding next step is to avoid host-mediated full-frontier exchange by using peer-to-peer or NCCL communication and a true distributed CSR layout. A follow-up experiment should also include weak scaling on generated RMAT graphs and compare with a tuned distributed GraphBLAS implementation.

11 CONCLUSION

We implemented and evaluated BFS across two parallel settings. On one A100, the fastest SpMV-style CUDA kernel outperformed our graph-first traversal on all current synthetic inputs, though graph-first remained close on random sparse/dense graphs. After the poster checkpoint, the MPI extension added a distributed-memory

perspective: 2D partitioning improved dense synthetic graphs at higher rank counts, while 1D bitmap reductions were stronger on sparse road networks. The optional multi-GPU prototype did not scale because per-level frontier synchronization dominated the saved row-scan work. Together, these experiments show why BFS remains a useful benchmark for parallel computing: the best algorithm is not simply the one with the most parallelism, but the one that best matches frontier structure to the memory and communication costs of the target machine.

ACKNOWLEDGMENTS

Experiments were run on NERSC Perlmutter. Source code, scripts, and benchmark outputs are available at <https://github.com/Harshaan-Chugh/parallel-bfs>.

REFERENCES

- [1] S. Beamer, K. Asanovic, and D. Patterson. Direction-optimizing breadth-first search. In *SC '12: Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, 2012.
- [2] D. Merrill, M. Garland, and A. Grimshaw. Scalable GPU graph traversal. In *PPoPP '12*, 2012.
- [3] J. Kepner, P. Aaltonen, D. Bader, A. Buluc, F. Franchetti, J. Gilbert, D. Hutchison, M. Kumar, A. Lumsdaine, H. Meyerhenke, S. McMillan, J. Moreira, J. Owens, C. Yang, M. Zalewski, and T. Mattson. Mathematical foundations of the GraphBLAS. In *IEEE HPEC*, 2016.
- [4] A. Buluc and J. R. Gilbert. The Combinatorial BLAS: Design, implementation, and applications. *International Journal of High Performance Computing Applications*, 25(4), 2011.
- [5] J. Leskovec and A. Krevl. SNAP datasets: Stanford large network dataset collection. <https://snap.stanford.edu/data/>, 2014.
- [6] S. Beamer, K. Asanovic, and D. Patterson. The GAP benchmark suite. *arXiv:1508.03619*, 2015.